



AI Assistant Module — Tài liệu Kiến trúc & Bảo vệ

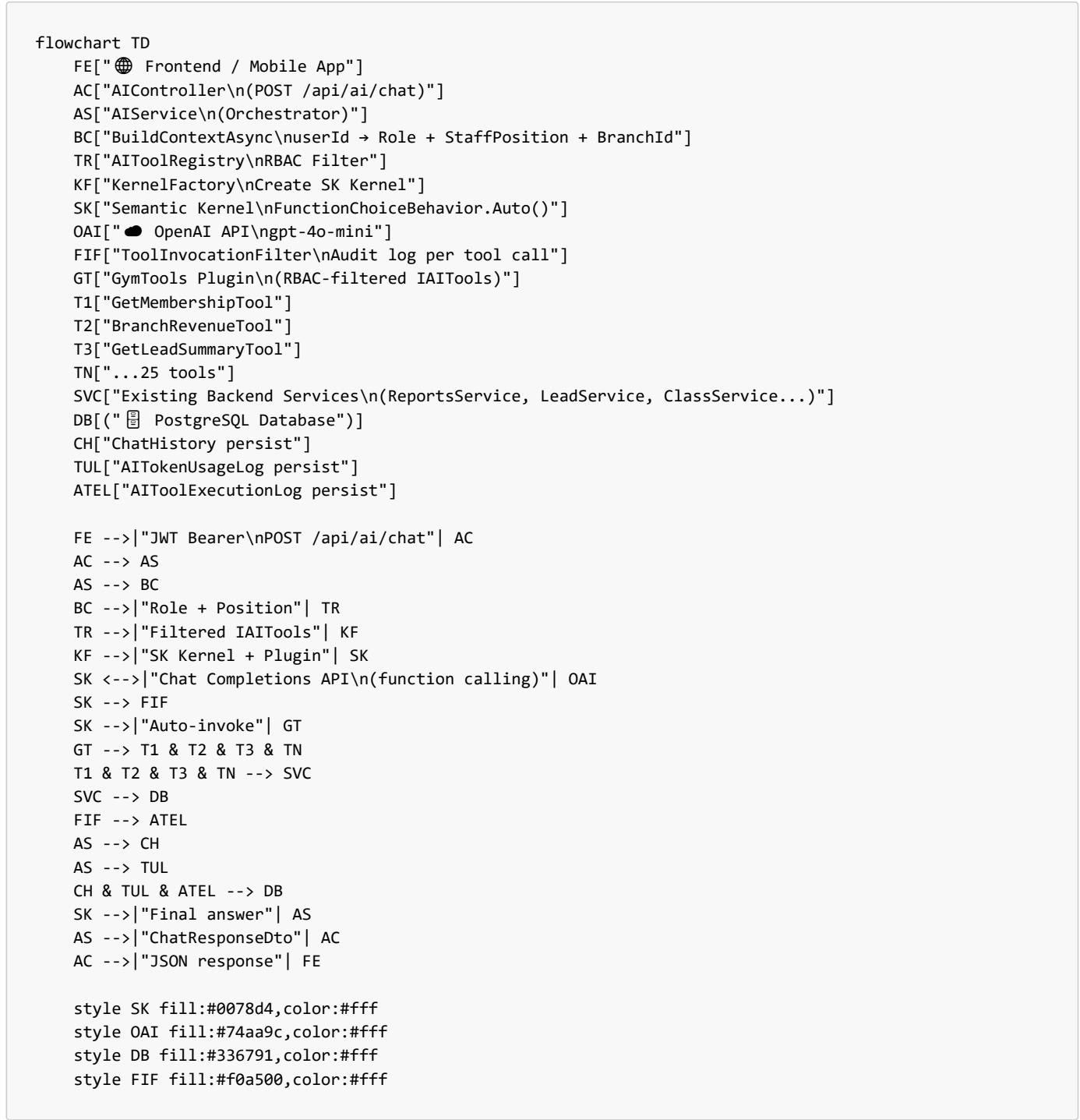
Phiên bản: 2.0 — Semantic Kernel Architecture

Framework: Microsoft Semantic Kernel v1.30.0 + ASP.NET Core 9

LLM: OpenAI GPT-4o-mini (mặc định) / Ollama (local)

1. Tổng quan kiến trúc

1.1 Sơ đồ kiến trúc tổng thể



1.2 Nguyên tắc thiết kế

Nguyên tắc	Hiện thực hóa
Separation of Concerns	IAITool = business/RBAC layer; SK = AI orchestration layer

Nguyên tắc	Hiện thực hóa
Single Responsibility	Mỗi tool xử lý 1 domain nghiệp vụ
Provider Agnostic	KernelFactory swap OpenAI ↔ Ollama qua config, không thay code
Security by Design	RBAC filter trước khi tools được đưa vào SK Kernel
Observability	Mỗi tool call được audit log; mỗi chat ghi token usage

2. Use Case Diagram

```
flowchart LR
    subgraph Actors
        MB["Member\n(Hội viên)"]
        SA["Sales\n(Nhân viên KD)"]
        PT["PT\n(Huấn luyện viên)"]
        RE["Receptionist\n(Lễ tân)"]
        BA["BranchAdmin\n(Quản lý CN)"]
        GO["GymOwner\n(Chủ phòng gym)"]
        SU["SuperAdmin\n(Quản trị hệ thống)"]
    end

    subgraph AI_System ["AI Assistant System"]
        UC1["Chat với AI Assistant"]
        UC2["Xem lịch sử chat"]
        UC3["Khám phá tools khả dụng"]
        UC4["Xem thống kê token usage"]
        UC5["Xem thông tin gói tập"]
        UC6["Xem/đặt lịch tập"]
        UC7["Nhận kế hoạch tập & dinh dưỡng"]
        UC8["Xem leads & phiếu bán hàng"]
        UC9["Tra cứu hợp đồng"]
        UC10["Xem danh sách lớp học / học viên"]
        UC11["Xem lịch dạy của PT"]
        UC12["Xem doanh thu chi nhánh"]
        UC13["Xem dashboard KPI toàn hệ thống"]
        UC14["Xem bảng lương nhân viên"]
    end

    MB --> UC1 & UC2 & UC3 & UC5 & UC6 & UC7
    SA --> UC1 & UC2 & UC3 & UC8 & UC9
    PT --> UC1 & UC2 & UC3 & UC10 & UC11
    RE --> UC1 & UC2 & UC3 & UC6 & UC9
    BA --> UC1 & UC2 & UC3 & UC8 & UC9 & UC10 & UC11 & UC12 & UC14
    GO --> UC1 & UC2 & UC3 & UC4 & UC12 & UC13 & UC14
    SU --> UC1 & UC2 & UC3 & UC4 & UC12 & UC13 & UC14
```

3. Đặc tả Use Case

UC1 — Chat với AI Assistant

Mục	Nội dung
Tên	Chat với AI Assistant
Actor	Member, Staff (Sales/PT/Receptionist/BranchAdmin), GymOwner, SuperAdmin
Mô tả	Người dùng gửi tin nhắn tự nhiên; AI tự động gọi tools phù hợp để lấy dữ liệu thực và trả lời
Điều kiện tiên quyết	Đã đăng nhập, có JWT token hợp lệ

Mục	Nội dung
Luồng chính	1. Người dùng gửi <code>POST /api/ai/chat</code> với <code>{ message }</code> 2. Hệ thống xác định Role + StaffPosition từ JWT 3. AI lọc tools theo quyền 4. LLM phân tích intent, chọn tool phù hợp 5. SK tự động gọi tool, nhận kết quả 6. LLM tổng hợp câu trả lời bằng tiếng Việt 7. Trả về <code>{ message, type }</code>
Luồng ngoại lệ	API Key lỗi → trả về thông báo lỗi cấu hình Tool lỗi → SK tiếp tục với thông báo lỗi, không crash
Hậu điều kiện	Chat message lưu vào <code>ChatHistory</code> ; token usage lưu vào <code>AITokenUsageLog</code>

UC3 — Khám phá tools khả dụng

Mục	Nội dung
Tên	Tool Discovery
Actor	Mọi role đã đăng nhập
Mô tả	Trả về danh sách tools mà người dùng được phép dùng, kèm JSON Schema tham số
Bảo mật	Member không thấy <code>get_branch_revenue</code> , <code>get_payroll_overview</code> — filtered hoàn toàn phía server
Endpoint	<code>GET /api/ai/tools</code>

UC4 — Xem thống kê token usage

Mục	Nội dung
Tên	Token Usage Statistics
Actor	SuperAdmin, GymOwner
Mô tả	Tổng token đã dùng trong N ngày, breakdown theo Role, ước tính chi phí USD
Endpoint	<code>GET /api/ai/usage?days=30</code>
Response	<code>{ totalPromptTokens, totalCompletionTokens, estimatedCostUsd, tokensByRole }</code>

UC7 — Nhận kế hoạch tập & dinh dưỡng (Member)

Mục	Nội dung
Tên	Generate Fitness Plan
Actor	Member
Mô tả	AI tạo kế hoạch tập cá nhân hóa dựa trên lịch sử, mục tiêu, gói tập
Tool	<code>generate_fitness_plan</code>
Hậu điều kiện	Kế hoạch lưu vào <code>AIRecommendation</code> ; truy xuất qua <code>GET /api/ai/recommendations</code>

4. Sequence Diagram — Chat có Tool Call

```
sequenceDiagram
    participant U as 👤 Người dùng
    participant AC as AIController
    participant AS as AIService
    participant TR as AIToolRegistry
    participant SK as Semantic Kernel
```

```
participant OAI as OpenAI API
participant TIF as ToolInvocationFilter
participant T as IAITool.ExecuteAsync
participant SVC as Backend Service
participant DB as Database

U->>AC: POST /api/ai/chat {message: "Doanh thu tháng này?"}
Note over AC: Validate JWT, extract userId
AC->>AS: HandleChatAsync(userId, request)

AS->>DB: Lookup userId → Role=Staff, Position=BranchAdmin
AS->>TR: GetAvailableTools(ctx)
Note over TR: RBAC filter: BranchAdmin → get_branch_revenue ✓
get_payroll_overview ✓, get_membership_info X (Member only)
TR-->>AS: List (filtered)

AS->>SK: Build Kernel + GymTools plugin (wrapped tools)
AS->>SK: Load chat history từ DB (last 20 messages)
AS->>SK: GetChatMessageContentAsync(settings=Auto)

SK->>OAI: Chat Completions API (functions: get_branch_revenue, ...)
OAI-->>SK: finish_reason: "tool_calls"
tool_call: get_branch_revenue({month:5, year:2026})

SK->>TIF: OnFunctionInvocationAsync (intercept)
TIF->>T: Wrapped KernelFunction → IAITool.ExecuteAsync(args, ctx)
T->>SVC: ReportsService.GetRevenueReportAsync(query, userId)
SVC->>DB: SELECT doanh thu tháng 5/2026
DB-->>SVC: RevenueReportDto
SVC-->>T: ToolResult.Ok("💰 Doanh thu 5/2026: 120,000,000 VNĐ...", report)
T-->>SK: "💰 Doanh thu 5/2026: 120,000,000 VNĐ..."
TIF->>DB: INSERT AIToolExecutionLog (tool=get_branch_revenue, success=true, 234ms)

SK->>OAI: Chat Completions API (tool result appended)
OAI-->>SK: finish_reason: "stop"
content: "Doanh thu tháng 5/2026 của chi nhánh đạt 120 triệu VNĐ..."

SK-->>AS: ChatMessageContent (finalText + Usage metadata)
AS->>DB: INSERT ChatHistory (user message + assistant response)
AS->>DB: INSERT AITokenUsageLog (847 prompt + 203 completion tokens)
AS-->>AC: ChatResponseDto { message, type="text" }
AC-->>U: 200 OK { data: { message: "Doanh thu tháng 5..." } }
```

5. Bảng phân quyền Tools (RBAC Matrix)

Roles & Positions

```
Application Roles (ASP.NET Identity):
  SuperAdmin | GymOwner | Staff | Member

StaffPosition (chỉ áp dụng khi Role = Staff):
  BranchAdmin | Sales | PT | HeadPT | Receptionist
```

Tool → Role/Position Mapping

Tool Name	Member	Sales	PT	HeadPT	Receptionist	BranchAdmin	GymOwner	SuperAdmin
get_membership_info	☑							
get_available_packages	☑							
get_my_schedule	☑							

Tool Name	Member	Sales	PT	HeadPT	Receptionist	BranchAdmin	GymOwner	SuperAdmin
book_class	☒							
cancel_booking	☒							
get_attendance_summary	☒							
generate_fitness_plan	☒							
ask_fitness_coach	☒							
get_lead_summary		☒				☒	☒	☒
get_lead_pipeline		☒				☒	☒	☒
get_sales_funnel		☒				☒	☒	☒
contract_lookup		☒			☒	☒	☒	☒
get_class_roster			☒	☒		☒	☒	☒
get_member_training_overview			☒	☒		☒	☒	☒
get_my_teaching_schedule			☒	☒				
get_pt_performance				☒		☒	☒	☒
checkin_lookup					☒	☒	☒	☒
booking_lookup					☒	☒	☒	☒
member_lookup					☒	☒	☒	☒
get_checkin_report						☒	☒	☒
get_branch_revenue						☒	☒	☒
get_branch_payroll						☒	☒	☒
get_branch_dashboard						☒	☒	☒
get_global_revenue							☒	☒
get_system_dashboard							☒	☒
get_payroll_overview							☒	☒
get_global_sales_funnel							☒	☒

6. Q&A Phản biện

? "Tại sao dùng Semantic Kernel thay vì tự viết?"

Semantic Kernel là enterprise AI orchestration framework của Microsoft, được dùng trong production bởi các công ty lớn. Nó cung cấp:

- **Auto function calling loop** — tự động xử lý vòng lặp LLM ↔ tool mà không cần viết thủ công
- **IFunctionInvocationFilter** — hook để audit mọi tool call, tích hợp observability
- **Provider abstraction** — swap OpenAI sang Azure OpenAI hay Ollama bằng 1 dòng config
- **Production-tested** — không phải reinvent the wheel với HTTP client, JSON parsing, parallel tool calls

Tự viết sẽ tạo ra technical debt và bug tiềm ẩn (ví dụ: sai format assistant message khi có tool_calls).

? "Nếu đã dùng Semantic Kernel thì IAITool để làm gì?"

Đây là **Separation of Concerns** có chủ đích:

- **IAITool** = business/RBAC layer — nơi đặt logic "ai được làm gì"
 - **AllowedRoles** → SuperAdmin, GymOwner, Staff, Member

- `AllowedStaffPositions` → BranchAdmin, Sales, PT, HeadPT, Receptionist
- `ExecuteAsync` → gọi existing backend services (không duplicate logic)

- **Semantic Kernel** = AI orchestration layer — nơi đặt logic "LLM gọi tool như thế nào"

SK không có khái niệm `StaffPosition`-based filtering. Nếu bỏ `IAITool`, phải nhúng RBAC logic vào SK internals — vi phạm Single Responsibility và rất khó test.

`SkToolHelper.WrapAsTool()` là bridge layer: convert `IAITool` → SK `KernelFunction` mà không thay đổi business logic.

? "Tại sao dùng `FunctionChoiceBehavior.Auto()` thay vì chỉ định tool cụ thể?"

`Auto()` cho phép LLM tự quyết định khi nào cần gọi tool và gọi tool nào. Đây là chuẩn của modern AI agent:

- LLM đủ thông minh để không gọi tool khi không cần (ví dụ: câu hỏi chào hỏi)
- LLM có thể gọi nhiều tools song song trong 1 lượt (parallel function calling)
- Không cần hardcode "câu hỏi về doanh thu → gọi tool X" như intent routing

? "Chi phí vận hành AI module như thế nào?"

Có thể xem real-time tại `GET /api/ai/usage?days=30`.

Ước tính với **gpt-4o-mini**:

- 1 chat turn trung bình: ~800 prompt tokens + 200 completion tokens = 1,000 tokens
- Chi phí: \$0.15/1M prompt + \$0.60/1M completion ≈ **\$0.00024/chat**
- 1,000 chats/tháng ≈ **\$0.24/tháng** — cực kỳ rẻ so với tính năng mang lại

? "Security — làm thế nào đảm bảo Member không dùng được admin tools?"

Ba lớp bảo vệ:

1. **RBAC filter trước SK** — `AIToolRegistry.GetAvailableTools(ctx)` chỉ trả về tools phù hợp role. Member không bao giờ nhận được `get_branch_revenue` trong danh sách → LLM không biết tool đó tồn tại → không thể invoke
2. **ResolveAuthorized trong Registry** — nếu LLM cố tình gọi tool không trong danh sách, registry trả về null và log `UNAUTHORIZED`
3. **JWT trên mọi request** — không có token hợp lệ = 401 Unauthorized từ middleware, không đến được `IService`

? "Hệ thống có thể mở rộng thêm LLM Provider khác không?"

Có. Chỉ cần thay đổi config `AI:Provider`:

- `OpenAI` → OpenAI API (default)
- `Ollama` → Ollama local server (OpenAI-compatible API)
- Thêm provider mới (Azure OpenAI, Anthropic) → thêm case trong `KernelFactory.Create()`, không đụng đến tools hay business logic

? "Tại sao không dùng Semantic Kernel Plugins với `[KernelFunction]` attribute trực tiếp?"

Hybrid Wrapper Pattern được chọn vì:

1. `[KernelFunction]` attribute gắn coupling SK vào business layer — nếu sau này đổi framework, phải rewrite toàn bộ tools
2. RBAC filtering (`AllowedRoles`, `AllowedStaffPositions`) không thể express bằng `[KernelFunction]` — cần metadata riêng trên `IAITool`
3. `SkToolHelper.WrapAsTool()` là thin bridge — 30 dòng code, convert `IAITool` → `KernelFunction` on-the-fly. Tools hoàn toàn independent với SK

7. Kết luận kiến trúc

Kế thừa từ existing backend (không duplicate):

- ☒ ReportsService, LeadService, ClassService, AttendanceService, PayrollService
- ☒ Repository pattern, EF Core, Identity

Thêm AI layer (clean architecture):

- ☒ IAITool → domain-organized, RBAC-aware, testable independently
- ☒ AIToolRegistry → centralized authorization, single responsibility
- ☒ Semantic Kernel → enterprise AI orchestration, provider agnostic
- ☒ ToolInvocationFilter → observability, audit trail
- ☒ AITokenUsageLog → cost monitoring, production readiness